Chapter 6. CPU scheduling. Full solutions

These original derivations correspond to the stable identifiers in the separate exercises PDF.

OS06-E01. Nonpreemptive scheduling

1. FCFS completes P1, P2, P3 at 8,12,14; waits are 0,8,12 and turnarounds 8,12,14. Mean waiting is 20/3. SJF executes P3, P2, P1, completing at 2,6,14 in execution order, with waits 0,2,6 and turnarounds 2,6,14. In process order P1, P2, P3, SJF completion is 14,6,2 and waiting is 6,2,0.
2. SJF mean waiting is 8/3 rather than 20/3, lower by 4 units. FCFS total turnaround is 34, mean 34/3; SJF total is 22, mean 22/3. The policies do not have equal total turnaround.
3. The actual next burst lengths must be known for the ideal ordering and simultaneous-arrival result. A prediction can be used operationally, but errors invalidate an unqualified claim about that ideal schedule.

Validation criterion. Align every metric with its process identity and time unit.

OS06-E02. Round Robin

1. P1[0,2), P2[2,4), P3[4,6), P1[6,8), P3[8,9), P1[9,10). Completion in process order is 10,4,9.
2. With no I/O, waiting equals completion minus burst, giving 5,2,6. First-dispatch response is 0,2,4. Mean waiting is 13/3, approximately 4.33.
3. With q=1, first responses become 0,1,2. The sequence is P1, P2, P3, P1, P2, P3, P1, P3, P1, P1 in unit intervals, so completions are P1=10, P2=5, P3=8. Waiting becomes 5,3,5. More slice boundaries can increase real switching overhead; a final consecutive P1 slice need not switch threads.

Validation criterion. Nonoverlapping intervals must account for all ten execution units, plus any explicitly included overhead.

Chapter 6. CPU scheduling. Full solutions (continued)

OS06-E03. Priorities and starvation

1. Show an admissible unbounded arrival sequence in which other work always wins and the batch job never progresses. One long observed delay is not proof of an infinite wait.
2. Aging increases effective priority with waiting. It can eventually overtake competitors only if the increase, saturation, tie-break, and admission rules allow that outcome.
3. Specify an aging function or reserved-service mechanism, ties, preemption, and worst-case arrivals. Derive the bound using those rules; an unspecified promise to age jobs is insufficient.

Validation criterion. Mean performance and a fairness guarantee are different claims.

OS06-E04 HRTICK CFS event reasoning

1. n=2≤8 gives period 6. Each slice=6×1024/2048=3 ms. Nice 0 adds one vruntime ms per real ms.
2. Post-selection states are 0 P1/{P2@0}/min 0; 3 P2/{P1@3}/min 0; 6 P1/{P2@3}/min 3; 9 P3/{P2@3, P1@6}/min 3; 11 P2/{P1@6}/min 3; 12 P1/{}/min 6; 14 none/{}/min 8. Ready excludes current; min never decreases. Equal keys follow the given insertion order.
3. HRTICK causes switches at 3/6/9, wake also selects P3 at 9; completions cause switches at 11/12. Thus P1[0,3), P2[3,6), P1[6,9), P3[9,11), P2[11,12), P1[12,14). Completion 14/12/11. At 8 there is no event satisfying a preemption condition.
4. P3 returns at max(0,3−3)=0. At 9 current P1 has vruntime 6, so lead 6>1 immediately selects P3. It cannot preempt at 8 before its wake.

OS06-E05 Strict periodic-tick comparison

1. delta_exec=3 at 3, 4 at 4. First condition is >ideal, so 3>3 is false and 4>3 true.
2. P1[0,4), P2[4,8) completes, P1[8,9), P3[9,11), P1[11,14). At 9 min 5 places P3 at 2 and P1 key 5 leads 3>1. Completion 14/8/11.
3. Ideal runtime is the scheduling slice, not a hardware timing promise. Ordinary tick resolution and its strict condition determine observation points. Queued HRTICK directly requests rescheduling at the exact scheduled boundary.
4. P1[6,9) first becomes [6,8.5). Then P3[8.5,10.5), P2[10.5,11.5), P1[11.5,14). Completions 14/11.5/10.5.

Chapter 6. CPU scheduling. Full solutions (continued)

OS06-E06 CPU fairness and synchronization

1. Total 1359. A slice 4.521 ms, B slice 1.479 ms; shares 1024/1359≈75.35%,335/1359≈24.65%.
2. No. Two ordinary counter++ operations can interleave loads and stores even on one preemptive CPU, losing an increment in the abstract model; in C/C++ the unsynchronized conflicting non-atomic accesses are a data race with undefined behavior. CPU share does not make RMW atomic.
3. Publish ready with release store and observe it with an acquire load reading that release, or use an equivalent mutex/CV protocol. A numeric ordinary ready=1 does not establish buffer visibility. Buffer lifetime and absence of conflicting subsequent mutations are also required.
4. The same mutex around every conflicting access excludes overlap. A lock/unlock API with acquire/release semantics also establishes the corresponding happens-before edge. Publication must actually occur inside that contract, with correct waits, errors, and releases.
5. A barrier orders operations but does not admit only one thread. An atomic operation is indivisible for the specified location; its memory-order contract can publish other data. A mutex owns an entire critical section, usually with lock-acquire/unlock-release ordering. A binary semaphore initially 1 can serialize through successful wait and authorized signal but has no owner; counting semaphores bound permits. Semaphore visibility depends on the API. Fair scheduling, atomicity, exclusion, and memory ordering are four distinct guarantees.

Chapter 6. CPU scheduling. Full solutions (continued)

Verification code appendix: scheduler_demo.py

```
"""Small deterministic CPU scheduling models.

Run:
    python scheduler_demo.py
"""

from dataclasses import dataclass


@dataclass(frozen=True)
class Job:
    name: str
    burst: int


def metrics(order: list[Job]) -> dict[str, float]:
    elapsed = 0
    waits: list[int] = []
    responses: list[int] = []
    turnarounds: list[int] = []
    first_start = {}
    for job in order:
        waits.append(elapsed)
        first_start[job.name] = elapsed
        elapsed += job.burst
        responses.append(first_start[job.name])
        turnarounds.append(elapsed)
    return {
        "avg_wait": sum(waits) / len(waits),
        "avg_response": sum(responses) / len(responses),
        "avg_turnaround": sum(turnarounds) / len(turnarounds),
    }


def round_robin(jobs: list[Job], quantum: int) -> dict[str, int]:
    remaining = {job.name: job.burst for job in jobs}
    elapsed = 0
    response: dict[str, int | None] = {job.name: None for job in jobs}
    completion: dict[str, int] = {}
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 6. CPU scheduling. Full solutions (continued)

```
    queue = list(jobs)
    while queue:
        job = queue.pop(0)
        if response[job.name] is None:
            response[job.name] = elapsed
        run = min(quantum, remaining[job.name])
        elapsed += run
        remaining[job.name] -= run
        if remaining[job.name] == 0:
            completion[job.name] = elapsed
        else:
            queue.append(job)
    return {
        "completion_P1": completion["P1"],
        "completion_P2": completion["P2"],
        "completion_P3": completion["P3"],
        "response_P3": int(response["P3"]),
    }


def main() -> None:
    jobs = [Job("P1", 8), Job("P2", 4), Job("P3", 2)]
    fcfs = metrics(jobs)
    sjf = metrics(sorted(jobs, key=lambda job: job.burst))
    rr = round_robin(jobs, quantum=2)
    assert round(fcfs["avg_wait"], 2) == 6.67
    assert round(sjf["avg_wait"], 2) == 2.67
    assert rr["completion_P1"] == 14
    assert rr["response_P3"] == 4
    print("FCFS", fcfs)
    print("SJF", sjf)
    print("RR", rr)


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study